Unit 3: Integrity, Authentication & Hash Functions

Maitri
Hingu

Unit 3: Integrity, Authentication & Hash Functions

Integrity, authentication, and hash functions are fundamental concepts in computer security and cryptography, crucial for ensuring the confidentiality, integrity, and authenticity of data and communication. Let's explore each of these concepts in more detail:

- **Integrity:**

  Integrity refers to the assurance that data has not been tampered with or altered in any unauthorized or unintended way during storage, transmission, or processing. Maintaining data integrity is essential to ensure the reliability and trustworthiness of information.

Unit 3: Integrity, Authentication & Hash Functions

- **Integrity:**

  To ensure data integrity, various techniques are employed, including:

  - **Checksum and Hash Functions:** These are used to generate fixed-size values (hashes) based on the data's content. If the data changes, the hash value will change, indicating potential tampering.
  - **Digital Signatures:** Cryptographic techniques are used to sign data, ensuring it has not been modified since it was signed by a trusted entity.
  - **Access Controls:** Restricting access to data and using permissions to prevent unauthorized modifications.

Unit 3: Integrity, Authentication & Hash Functions

- **Authentication:**

  Authentication is the process of verifying the identity of a user, system, or entity to ensure that they are who they claim to be. Authentication mechanisms are vital to prevent unauthorized access and maintain the security of systems and data. Common methods of authentication include:

  - **Passwords:** Users provide a secret password that must match a stored value.
  - **Multi-Factor Authentication (MFA):** Requires multiple forms of authentication, such as a password and a one-time code sent to a mobile device.

Unit 3: Integrity, Authentication & Hash Functions

- **Authentication:**
  - **Biometrics:** Uses unique physical or behavioral traits, like fingerprints or facial recognition, for identity verification.
  - **Certificates:** Digital certificates issued by trusted authorities to verify the identity of entities in a secure manner.

Unit 3: Integrity, Authentication & Hash Functions

- **Hash Functions:**

  A hash function is a mathematical function that takes an input (or message) and produces a fixed-length string of characters, which is typically a hexadecimal number. The output, known as the hash value or digest, is unique to the input data. Hash functions have several important properties:
  - **Deterministic:** The same input will always produce the same hash value.
  - **Fast Computation:** Hash functions should be computationally efficient.

Unit 3: Integrity, Authentication & Hash Functions

- **Hash Functions:**
  - **Pre-image Resistance:** It should be computationally infeasible to reverse the process and derive the original input from the hash value.
  - **Collision Resistance:** It should be unlikely for two different inputs to produce the same hash value.

Unit 3: Integrity, Authentication & Hash Functions

- Hash functions are used in various security applications, including data integrity verification, password storage (hashing passwords instead of storing plaintext), and digital signatures. They help ensure data integrity and provide a means to verify the authenticity of data.
- In summary, integrity, authentication, and hash functions are integral components of computer security, working together to safeguard data, verify identities, and protect against unauthorized access or tampering. These concepts are foundational in building secure and trustworthy computing systems.

Unit 3: Integrity, Authentication & Hash Functions

Authentication Requirements and its Functions

- **Authentication Requirements:**
- **Identification:** Users must provide identification, such as a username or account number.
- **Authentication Factors:** Authentication involves factors like something you know (password), have (token), or are (biometrics).
- **Authentication Protocols:** Secure protocols are needed for exchanging and verifying credentials.
- **Secure Storage:** Protect user credentials through secure storage and hashing.

Unit 3: Integrity, Authentication & Hash Functions

Authentication Requirements and its Functions

- **Account Policies:** Implement policies for password strength and account lockout.
- **Multi-Factor Authentication (MFA):** Use multiple factors for stronger security.
- **Session Management:** Manage active sessions securely.
- **Monitoring and Logging:** Monitor authentication events for security insights.
- **User Education:** Educate users about secure authentication practices.
- **Compliance:** Comply with industry-specific regulations.

Unit 3: Integrity, Authentication & Hash Functions

Authentication Requirements and its Functions

- **Authentication Functions:**
- **Identity Verification:** Verify the claimed identity of users or entities.
- **Access Control:** Control access to resources based on authentication.
- **Accountability:** Link actions to authenticated identities for auditing.
- **Protection Against Unauthorized Access:** Prevent unauthorized access to systems and data.

Unit 3: Integrity, Authentication & Hash Functions

Message Authentication

- Message authentication, often referred to as message authentication code (MAC) or message integrity check (MIC), is a security mechanism used to verify the authenticity and integrity of a message or data transmitted between two parties. It ensures that the received message has not been tampered with during transit and that it indeed originated from the expected sender.

Unit 3: Integrity, Authentication & Hash Functions

Components of Message Authentication:

- **Message:** This is the data or message that needs to be authenticated.
- **Secret Key:** Both the sender and receiver share a secret key, which is used to generate and verify the authentication code.
- **Authentication Code:** The authentication code, also known as the MAC (Message Authentication Code) or MIC (Message Integrity Check), is a short piece of data generated by applying a cryptographic algorithm (such as HMAC - Hash-Based Message Authentication Code) to the message and the secret key.

Unit 3: Integrity, Authentication & Hash Functions

Process of Message Authentication

- **Sender's Side:**
  - The sender takes the message and the shared secret key.
  - Using a cryptographic algorithm, the sender generates an authentication code (MAC) by combining the message and the secret key.
  - The sender attaches the authentication code to the message.
- **Transmission:**
  - The sender transmits the message and the attached authentication code to the receiver over an distrusted communication channel.

Unit 3: Integrity, Authentication & Hash Functions

Process of Message Authentication

- **Receiver's Side:**
  - The receiver receives the message and the authentication code.
  - The receiver shares the same secret key with the sender.
  - The receiver uses the same cryptographic algorithm and the shared secret key to calculate an authentication code for the received message.
- **Verification:**
  - The receiver compares the authentication code calculated locally with the authentication code received from the sender.
  - If the two codes match, the receiver can trust the integrity and authenticity of the message.

Unit 3: Integrity, Authentication & Hash Functions

Benefits of Message Authentication

- **Data Integrity:** Message authentication ensures that the message has not been altered during transmission. If any part of the message is modified, the authentication code will not match, and the receiver will know that tampering has occurred.
- **Authentication:** It verifies the identity of the sender. Since the secret key is known only to the sender and receiver, a valid authentication code indicates that the message came from the expected sender.
- **Data Source Verification:** It helps ensure that the message was generated by a legitimate source, preventing impersonation or unauthorized access.

Unit 3: Integrity, Authentication & Hash Functions

Benefits of Message Authentication

- **Data Freshness:** Some message authentication protocols also include a timestamp to ensure that the message is not an old or replayed message.

Message authentication is a crucial security measure in various communication protocols and applications, including secure messaging, network communication, and data transfer over the internet, to guarantee the trustworthiness and authenticity of transmitted data.

Unit 3: Integrity, Authentication & Hash Functions

Message Authentication Codes

Message Authentication Codes (MACs) are cryptographic techniques used to provide message authentication and data integrity. They are commonly employed to verify the authenticity and integrity of data transmitted between parties in a secure communication system. MACs are particularly useful when sensitive information needs to be protected against unauthorized modification or tampering during transmission.

Unit 3: Integrity, Authentication & Hash Functions

Message Authentication Codes

- **Shared Secret Key:** MACs rely on a shared secret key that is known only to the sender and the receiver. This secret key is used to both generate and verify the MAC.
- **Cryptographic Hash Function:** MACs often utilize cryptographic hash functions (e.g., HMAC - Hash-Based Message Authentication Code) as the core mathematical operation to produce the authentication code. These hash functions take the message and the secret key as input to generate a fixed-size code.

Unit 3: Integrity, Authentication & Hash Functions

Message Authentication Codes

- **Authentication Code Generation:**
  - The sender takes the message and the shared secret key.
  - Using a MAC algorithm (which includes the hash function), the sender computes the MAC, which is a fixed-length code unique to the combination of the message and the secret key.
  - This MAC is then appended to the message.
- **Transmission:** The sender transmits both the message and the MAC to the receiver.

Unit 3: Integrity, Authentication & Hash Functions

Message Authentication Codes

- **Authentication Code Verification:**
- The receiver receives the message and the MAC.
- It knows the shared secret key.
- The receiver uses the same MAC algorithm and key to recalculate the MAC for the received message.
- If the recalculated MAC matches the received MAC, the message is considered authentic and unaltered. If there is a mismatch, it indicates tampering or a potential security breach.

Unit 3: Integrity, Authentication & Hash Functions

Hash Functions

Hash functions are fundamental cryptographic algorithms that take an input (or message) and produce a fixed-size string of characters, known as a hash value or hash code. These hash values are unique to the input data, and even a small change in the input results in a significantly different hash value. Hash functions serve various purposes in computer science, including data integrity verification, password storage, and data indexing.

Unit 3: Integrity, Authentication & Hash Functions

Hash Functions

- **Characteristics of Hash Functions:**
- **Deterministic:** For the same input, a hash function will always produce the same hash value.
- **Fixed Output Size:** Hash functions generate hash values of a fixed length, regardless of the input size.
- **Efficient:** Hash functions are computationally efficient and produce hash values quickly.
- **Pre-image Resistance:** Given a hash value, it should be computationally infeasible to determine the original input.
- **Collision Resistance:** It should be highly unlikely for two different inputs to produce the same hash value.

MAC V/S Hash Fucntion

MAC

- MACs are designed specifically for message authentication. They are used to verify the integrity and authenticity of a message and ensure it hasn't been tampered with during transmission.
- MACs require a secret key that is shared between the sender and receiver. The key is used to generate and verify the MAC. MACs are symmetric, meaning the same key is used for both generation and verification.

Hash Functions

- Hash functions have a broader range of applications, including data integrity verification, password storage, data structures, and more. They are not designed specifically for authentication but can be used in various security and non-security contexts.
- Hash functions do not require a secret key. They produce a hash value based solely on the input data and do not involve key-based encryption.

MAC V/S Hash Fucntion

MAC

- MACs are designed with security as their primary goal. They are constructed to resist various cryptographic attacks, and the use of a secret key enhances security.
- MACs are designed to be collision-resistant, meaning it should be computationally infeasible to find two different inputs that produce the same MAC.

Hash Functions

- While hash functions offer some level of security, they are not specifically designed for authentication. Hash functions like MD5 and SHA-1 are now considered insecure for cryptographic purposes due to vulnerabilities like collision attacks.
- Hash functions are also designed to be collision-resistant. However, some older hash functions (e.g., MD5, SHA-1) have been found to be susceptible to collision attacks.

MAC V/S Hash Fucntion

MAC

- MACs are primarily used for authentication, ensuring that the message came from a trusted sender and has not been altered.
- MACs are keyed, meaning they involve a secret key in the computation of the authentication code.

Hash Functions

- Hash functions are often used for data integrity verification, where the focus is on detecting data tampering rather than verifying the sender's identity.
- Hash functions are unkeyed and operate solely on the input data without the involvement of secret keys.

Unit 3: Integrity, Authentication & Hash Functions

MD5 Algorithm

MD5 (Message Digest Algorithm 5) is a widely used cryptographic hash function that was designed by Ronald Rivest in 1991. It is part of the MD (Message Digest) family of hash functions and is known for its speed and efficiency in generating fixed-size hash values from variable-length input data.

Unit 3: Integrity, Authentication & Hash Functions

MD5 Algorithm Key Characteristics

- **Output Length:** MD5 produces a 128-bit (16-byte) hash value.
- **Fixed Output Size:** Regardless of the size of the input data, MD5 always generates a 128-bit hash value.
- **Deterministic:** For the same input, MD5 will always produce the same hash value.
- **Pre-image Resistance:** It should be computationally infeasible to reverse the process and determine the original input from the MD5 hash value. However, MD5 is no longer considered secure in this aspect due to advances in computational power and cryptographic research.

Unit 3: Integrity, Authentication & Hash Functions

MD5 Algorithm Key Characteristics

- **Collision Resistance:** Collision resistance means it should be highly improbable to find two different inputs that produce the same MD5 hash value. Unfortunately, collision attacks against MD5 have been demonstrated, making it unsuitable for security-critical applications.

Unit 3: Integrity, Authentication & Hash Functions

MD5 Algorithm Operations

The MD5 algorithm processes input data in 512-bit (64-byte) blocks and produces a 128-bit (16-byte) hash value. Here is a simplified overview of how MD5 operates:

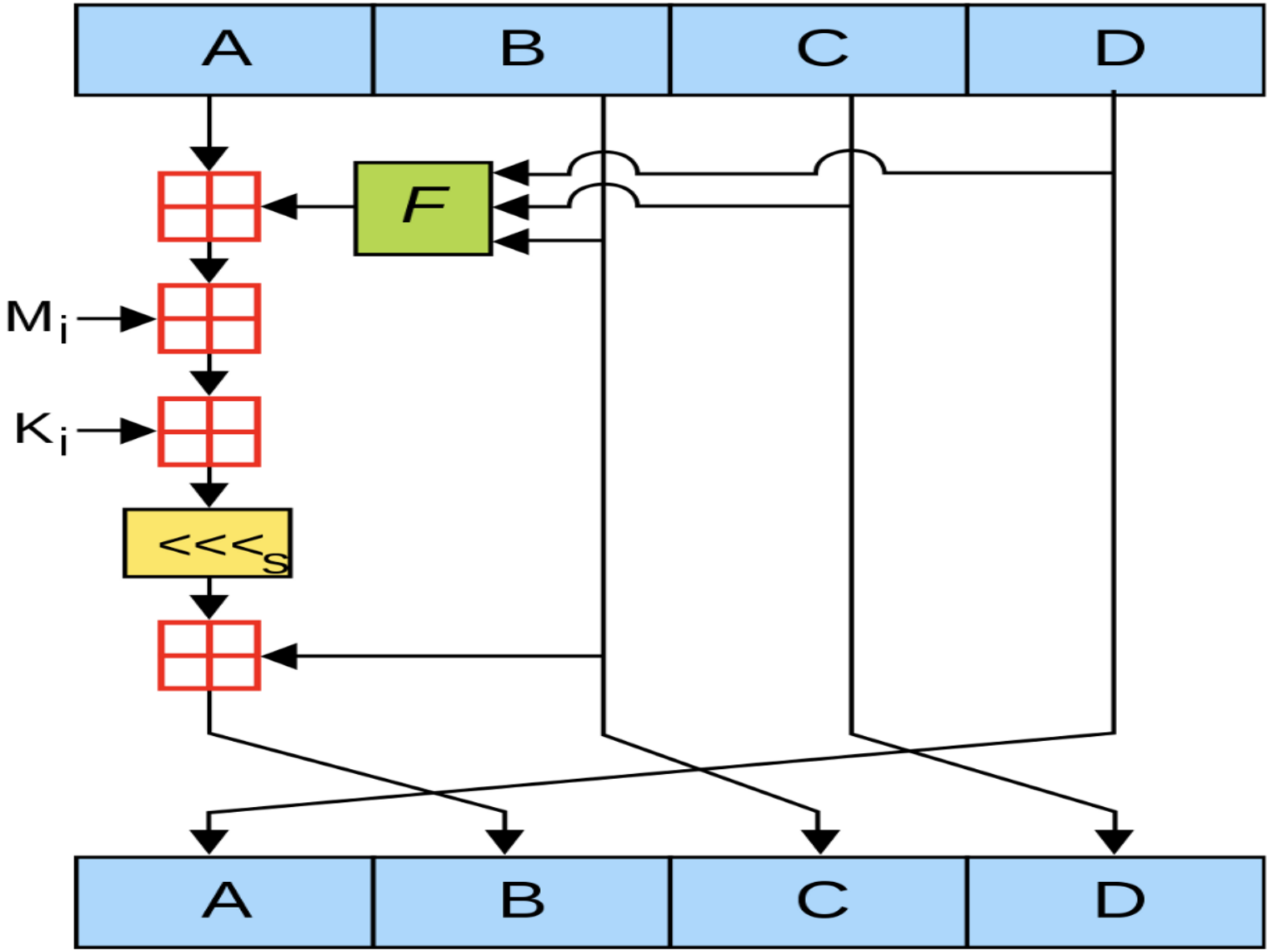
- **Padding:** If necessary, the input data is padded to ensure its length is a multiple of 512 bits.
- **Message Parsing:** The padded message is divided into 512-bit blocks.
- **Initialization:** MD5 uses four 32-bit words (A, B, C, and D) as internal variables. These are initialized with predefined constants.

Unit 3: Integrity, Authentication & Hash Functions

MD5 Algorithm Operations

- **Four Rounds:** MD5 consists of four rounds, each consisting of multiple operations, including bitwise operations (AND, OR, XOR), addition modulo 2^32, and logical functions.
- **Processing Blocks:** Each block of the message is processed through the four rounds, with the internal state being updated at each step.
- **Final Hash Value:** After processing all blocks, the final hash value is derived from the internal state of A, B, C, and D. The hash value is typically represented as a 32-character hexadecimal number.

MD5



Unit 3: Integrity, Authentication & Hash Functions

MD5 Algorithm Security Considerations

MD5 was once widely used for data integrity and password storage, but it is no longer considered secure for cryptographic applications due to vulnerabilities. Research has demonstrated practical collision attacks against MD5, where two different inputs produce the same hash value. This undermines its security for tasks like message integrity verification and digital signatures.

Unit 3: Integrity, Authentication & Hash Functions

MD5 Algorithm Security Considerations

As a result of these vulnerabilities, MD5 has been largely replaced by more secure hash functions, such as those in the SHA-2 family (e.g., SHA-256), for security-critical applications. MD5 is still used in some non-cryptographic contexts, like checksumming files for data integrity checks, but it should be avoided in favor of more secure alternatives for any security-related tasks.

Unit 3: Integrity, Authentication & Hash Functions

SHA (Secure Hash Algorithm)

The Secure Hash Algorithm (SHA) is a family of cryptographic hash functions designed to take an input (or message) and produce a fixed-size output (hash value) that is typically a string of numbers and letters. These hash functions play a crucial role in ensuring data integrity and security in various applications, such as digital signatures, password storage, and data verification.

Unit 3: Integrity, Authentication & Hash Functions

SHA (Secure Hash Algorithm)

- **SHA-0:**
- SHA-0 was the first version of the Secure Hash Algorithm, developed by the National Security Agency (NSA) in the United States in 1993.
- It produced a 160-bit hash value.
- However, it was quickly found to have security vulnerabilities, and it was never widely adopted.

Unit 3: Integrity, Authentication & Hash Functions

SHA (Secure Hash Algorithm)

- **SHA-1:**
- SHA-1 was introduced as a revision to SHA-0 in 1995.
- It also produced a 160-bit hash value.
- SHA-1 became widely used for many years in various security applications, but over time, researchers discovered significant vulnerabilities.
- In the mid-2000s, practical collision attacks against SHA-1 were demonstrated, which raised concerns about its security.
- Due to these vulnerabilities, SHA-1 is considered deprecated and is no longer recommended for secure applications.

Unit 3: Integrity, Authentication & Hash Functions

SHA (Secure Hash Algorithm)

- **SHA-2:**
- SHA-2 was designed by the NSA in 2001 to address the security issues of SHA-1.
- It includes several hash functions with different output sizes, such as SHA-224, SHA-256, SHA-384, and SHA-512, producing hash values of 224, 256, 384, and 512 bits, respectively.
- SHA-256, in particular, is widely used and considered secure for various cryptographic purposes.
- SHA-2 is currently the most commonly used family of hash functions in cryptographic applications.

Unit 3: Integrity, Authentication & Hash Functions

SHA (Secure Hash Algorithm)

- **SHA-3:**
- SHA-3 is the latest member of the Secure Hash Algorithm family, designed by Keccak and selected as the winner of the NIST (National Institute of Standards and Technology) HASH function competition in 2012.
- It offers hash functions with different output sizes, including SHA3-224, SHA3-256, SHA3-384, and SHA3-512.
- SHA-3 is based on a fundamentally different design approach compared to SHA-2, which adds diversity to the available cryptographic options.

Unit 3: Integrity, Authentication & Hash Functions

SHA (Secure Hash Algorithm)

- Each SHA algorithm goes through a rigorous process of analysis and evaluation by the cryptographic community and standards organizations to ensure its security and suitability for various applications. As security threats evolve, so do cryptographic standards, and researchers continue to explore new hash functions and improvements to existing ones to stay ahead of potential vulnerabilities. It's important to choose the appropriate SHA algorithm based on your specific security requirements and the latest recommendations from trusted sources like NIST.

Unit 3: Integrity, Authentication & Hash Functions

Working of SHA (Secure Hash Algorithm)

- The exact internal process varies between different SHA variants, but here are simplified overview of the general steps involved in the hash computation process.
- **Message Padding:**
- The input message is first padded to a multiple of the block size (e.g., 512 bits for SHA-256) to ensure it can be processed in fixed-size blocks.
- Padding typically includes appending a 1-bit followed by zeros and then adding the length of the original message in bits.

Unit 3: Integrity, Authentication & Hash Functions

Working of SHA (Secure Hash Algorithm)

- **Message Chunking:**
- The padded message is divided into fixed-size blocks or chunks. Each chunk is processed one at a time.
- **Initialization:**
- The hash function starts with an initial state (often called the "IV" or Initialization Vector), which is predefined and unique for each SHA variant.

Unit 3: Integrity, Authentication & Hash Functions

Working of SHA (Secure Hash Algorithm)

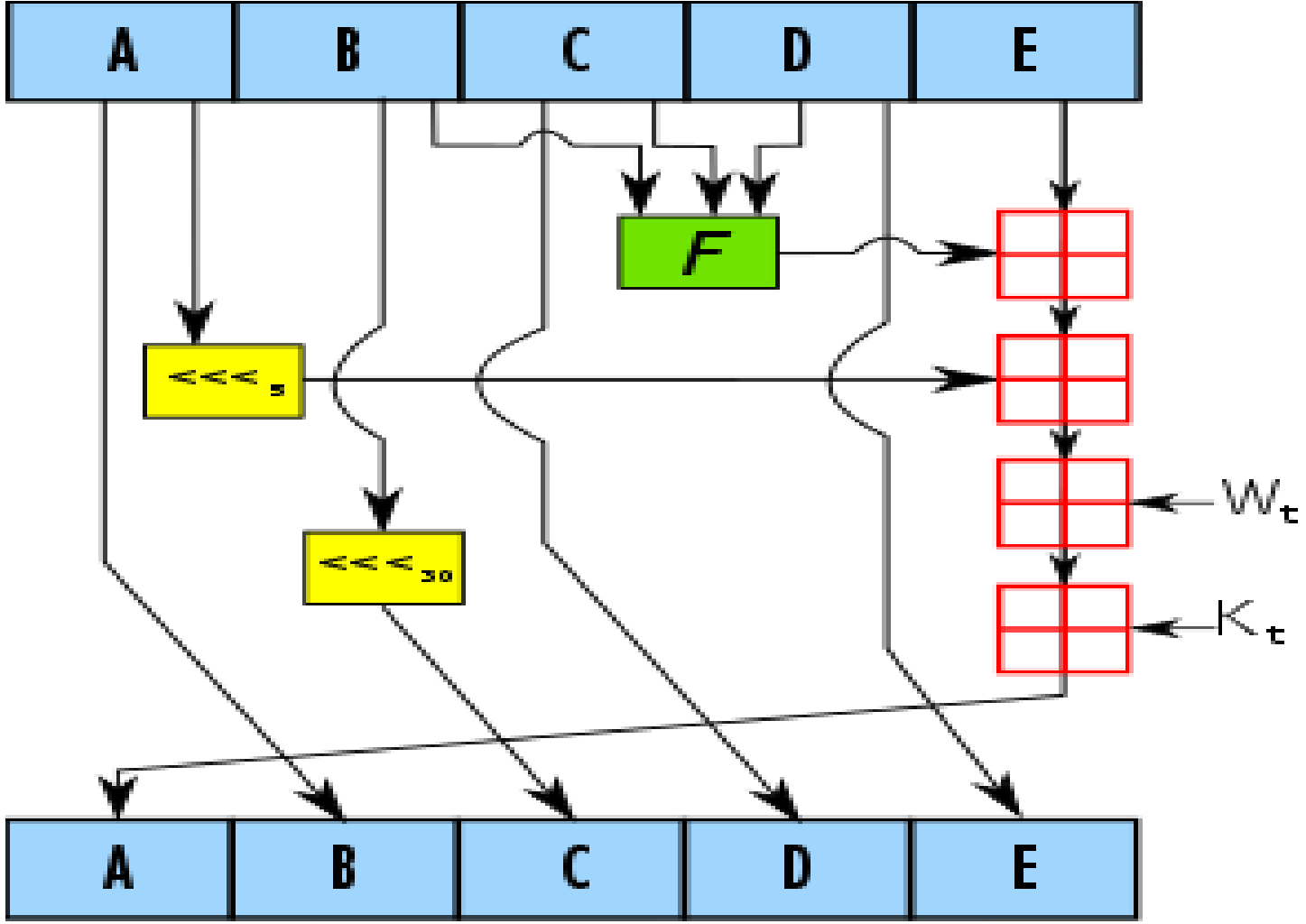
- **Compression Function:**
- Each message chunk is processed through a compression function, which combines the current state with the message chunk's data to update the state.
- The compression function typically involves multiple rounds of operations, such as bitwise operations (AND, OR, XOR), modular addition, logical functions (NOT, AND, OR), and bit rotation.

Unit 3: Integrity, Authentication & Hash Functions

Working of SHA (Secure Hash Algorithm)

- **Finalization:**
- After processing all chunks, the final state of the hash function represents the hash value of the entire input message.
- **Output:**
- The final state is usually transformed into the hash value format (e.g., converting to hexadecimal) to produce the hash output.

SHA-1



Unit 3: Integrity, Authentication & Hash Functions

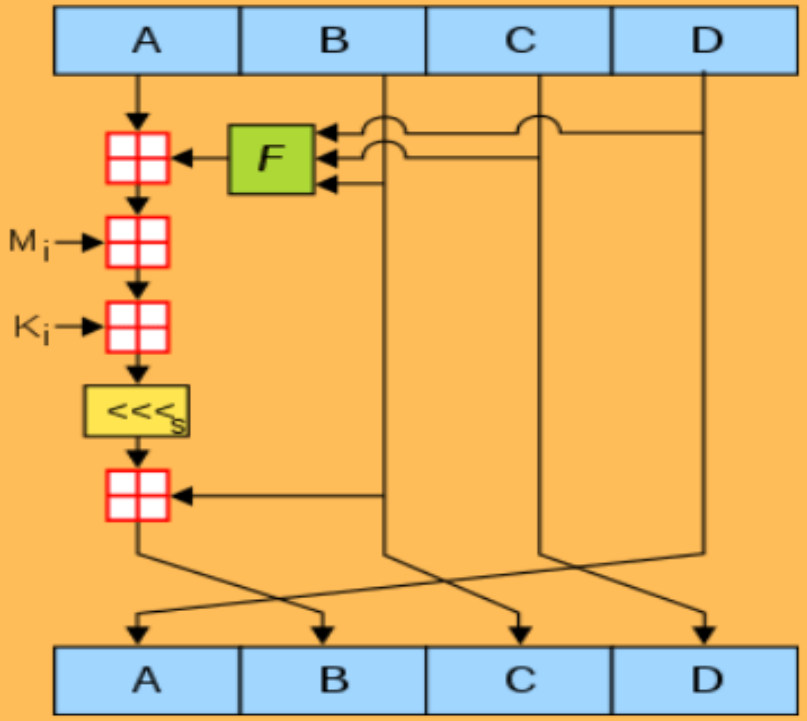
SHA (Secure Hash Algorithm)

- **Key Points:**
- **SHA-1** is considered deprecated and insecure due to vulnerabilities like collision attacks.
- **SHA-2** is widely used and includes variants like SHA-224, SHA-256, SHA-384, and SHA-512, each with different output sizes.
- **SHA-3** is a more recent addition to the family and is based on a different design approach (Keccak) than SHA-1 and SHA-2. It offers variable output sizes and is considered secure.

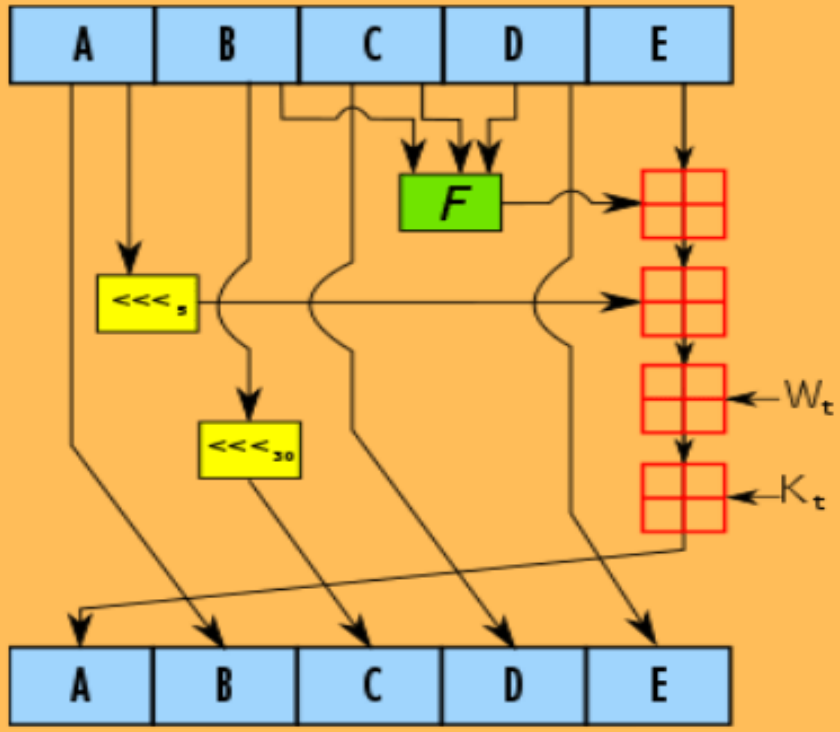
Unit 3: Integrity, Authentication & Hash Functions

SHA (Secure Hash Algorithm) Usage

- Data integrity verification.
- Secure password storage.
- Digital signatures for message authenticity.
- Cryptographic protocols like SSL/TLS.
- Efficient file deduplication.
- Data backup integrity checks.
- Digital forensics evidence verification.
- File download integrity via checksums.
- Blockchain technology for transaction security.
- Password hashing and verification.
- Software integrity verification.



MD5



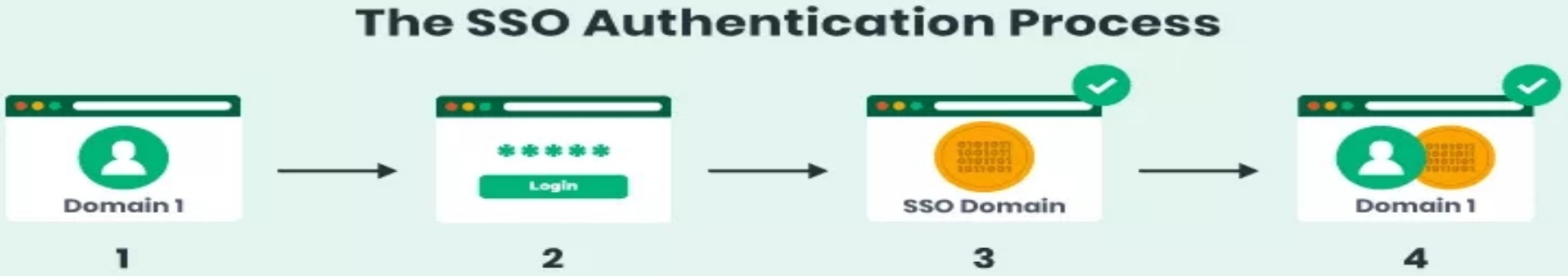
SHA-1

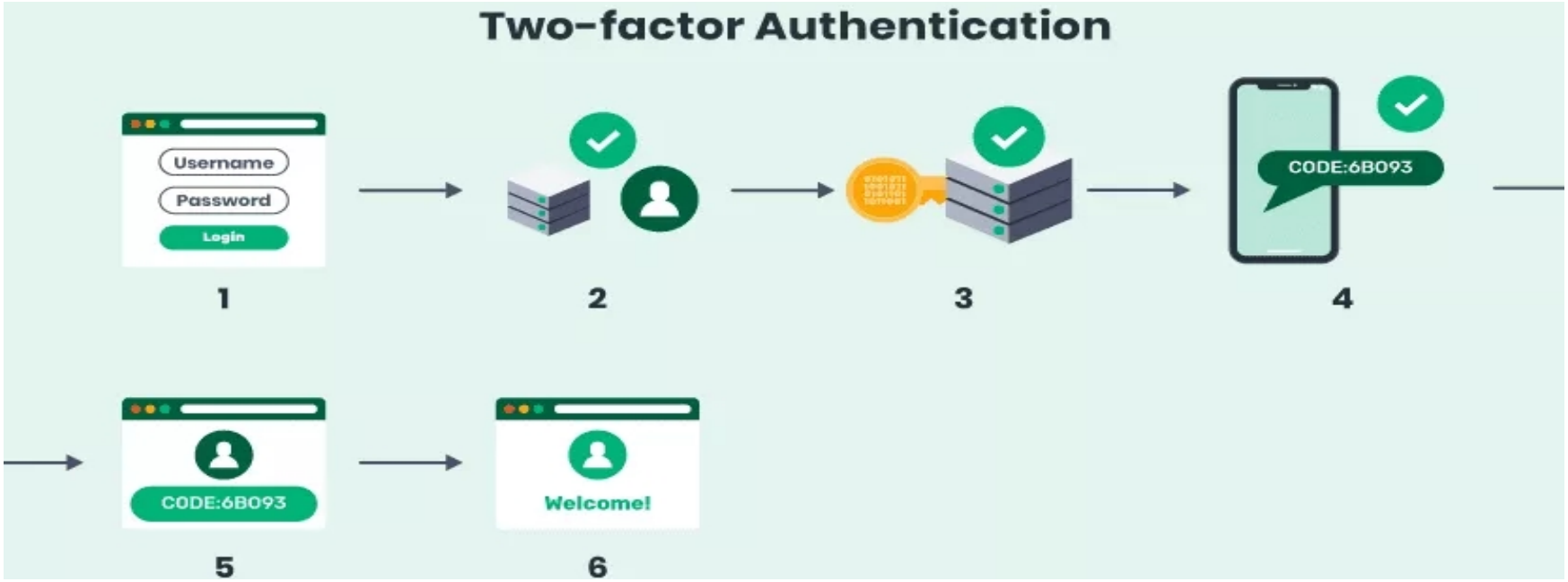
Unit 3: Integrity, Authentication & Hash Functions

User Authentication

- User authentication is a critical aspect of computer security that ensures that only authorized users can access a system or resource. It involves verifying the identity of a user, typically through a username and password or some other authentication method.
- User authentication refers to the process of verifying the identity of a user before granting access to a system, application, or resource. It is a fundamental security measure used to prevent unauthorized access to sensitive information.

Unit 3: Integrity, Authentication & Hash Functions


The SSO Authentication Process
Domain 1
*****
Login
SSO Domain
Domain 1
1
2
3
4


Two-factor Authentication
Username
Password
Login
CODE:6B093
CODE:6B093
Welcome!
1
2
3
4
5
6

Unit 3: Integrity, Authentication & Hash Functions

Remote User Authentication Principles

Remote user authentication principles outline the fundamental concepts and best practices for verifying the identity of users who are attempting to access a system or service remotely (i.e., over a network).

- **Authentication Factors:** Users typically provide one or more authentication factors, which can be something they know (e.g., a password), something they have (e.g., a smart card), or something they are (e.g., biometric data like fingerprints).

Unit 3: Integrity, Authentication & Hash Functions

Remote User Authentication Principles

- **Multi-Factor Authentication (MFA):** MFA involves using two or more authentication factors to enhance security. For example, a user might need to enter a password and then provide a fingerprint scan.
- **Security Protocols:** Authentication often relies on security protocols and standards to ensure that the exchange of authentication information is secure. Examples include TLS/SSL for secure communication and OAuth (Open Authorization) for authorization.
- **Password Policies:** Establishing password policies, such as requiring strong passwords and regular password changes, is crucial for protecting against password-based attacks.

Unit 3: Integrity, Authentication & Hash Functions

Remote User Authentication using Symmetric Encryption

Remote user authentication using symmetric encryption is a method of verifying a user's identity by encrypting and exchanging a shared secret between the user and the system.

- **Shared Secret:** Both the user and the system share a secret key or password in advance. This secret is known only to the user and the system.
- **Authentication Process:** When the user wants to access the system remotely, they send an authentication request along with their identity (e.g., username) and some encrypted data (often a nonce) to the system.

Unit 3: Integrity, Authentication & Hash Functions

Remote User Authentication using Symmetric Encryption

- **Encryption:** The system uses the shared secret to encrypt the data sent by the user. If the user's data can be successfully decrypted using the shared secret, it proves that the user possesses the secret and is authenticated.
- **Response:** If the decryption is successful, the system grants access to the user. Otherwise, access is denied.

This method relies on the secrecy of the shared secret key to ensure that only authorized users can decrypt the data.

Unit 3: Integrity, Authentication & Hash Functions

Kerberos

- Kerberos is a widely used network authentication protocol that provides strong authentication and secure communication in a distributed computing environment. It was developed at MIT and has become a standard for authenticating users and services on a network.

- **Ticket-Granting Ticket (TGT):** In Kerberos, users authenticate themselves to a Key Distribution Center (KDC) to obtain a Ticket-Granting Ticket (TGT). The TGT is a time-limited credential that proves the user's identity.

Unit 3: Integrity, Authentication & Hash Functions

Kerberos

- **Service Tickets:** When a user wants to access a network service (e.g., a file server), they request a service ticket from the KDC. The service ticket is encrypted using a session key derived from the TGT.
- **Authentication:** The user presents the service ticket to the desired service, which can decrypt it using its own secret key. If the service can successfully decrypt the ticket, it knows that the user is authenticated, and access is granted.
- **Session Keys:** Kerberos generates session keys for secure communication between the user and the service. These keys are used to encrypt and decrypt data exchanged during the session.

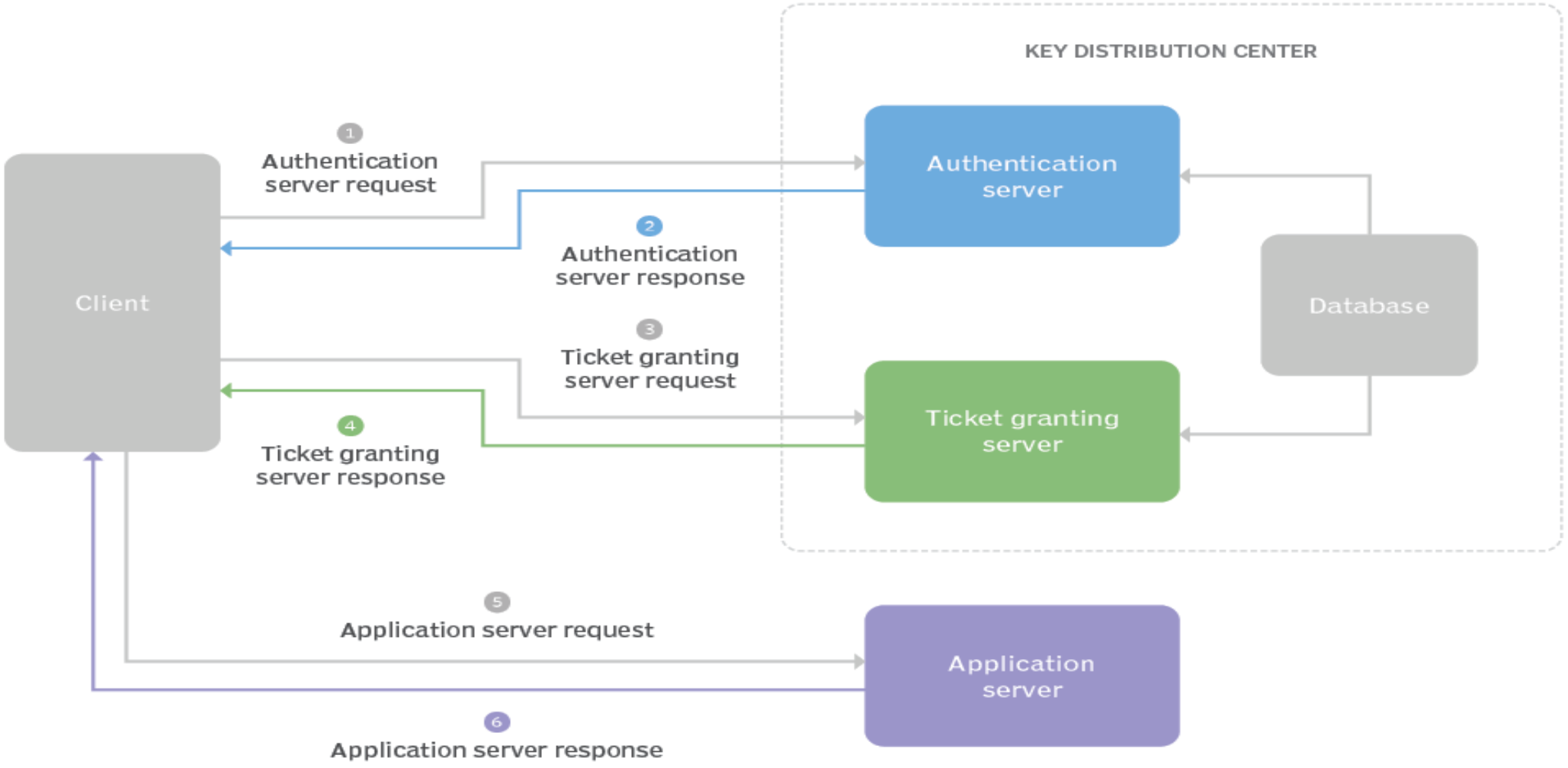
Unit 3: Integrity, Authentication & Hash Functions

Kerberos

Kerberos offers strong security because it relies on shared secrets (passwords) and uses encryption to protect authentication information. It's widely used in environments like Active Directory for Windows authentication and is a robust solution for network authentication and security.

In summary, remote user authentication is a crucial aspect of securing computer systems, and methods like symmetric encryption and protocols like Kerberos play a significant role in achieving this security. These methods ensure that only authorized users can access resources while protecting authentication information from unauthorized access.

The Kerberos authentication process



Unit 3: Integrity, Authentication & Hash Functions

Digital Signatures and Authentication Protocols

Digital signatures and authentication protocols are essential components of information security, helping to verify the authenticity and integrity of data and ensuring that parties involved in communication are who they claim to be.

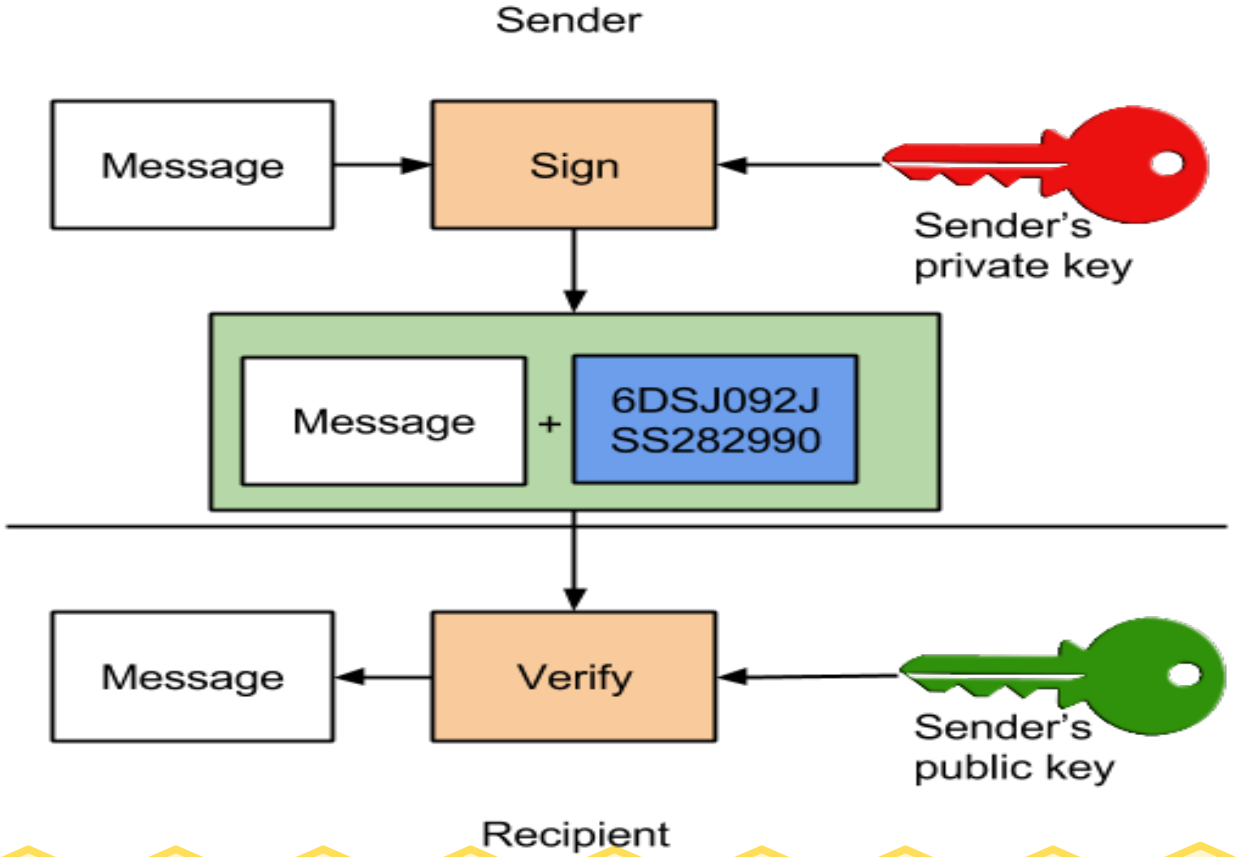
Unit 3: Integrity, Authentication & Hash Functions

Unit 3: Integrity, Authentication & Hash Functions

Introduction to Digital Signatures

- A digital signature is a cryptographic technique used to ensure the authenticity, integrity, and non-repudiation of digital messages or documents. It provides a way for the recipient of a message to verify that it was indeed created by the claimed sender and that it hasn't been tampered with during transmission.

Unit 3: Integrity, Authentication & Hash Functions



Unit 3: Integrity, Authentication & Hash Functions

Working of Digital Signatures

- **Key Pair:** In digital signature schemes, entities (typically users or organizations) have a pair of cryptographic keys: a private key and a public key. The private key is kept secret, while the public key is shared with others.
- **Signing:** To create a digital signature, the sender uses their private key to perform a mathematical operation on the message or document they want to sign. This operation generates a unique signature.

Unit 3: Integrity, Authentication & Hash Functions

Working of Digital Signatures

- **Verification:** The recipient of the message uses the sender's public key to verify the digital signature. If the signature can be successfully verified, it means that the message hasn't been altered since it was signed, and it was indeed signed by the holder of the private key.
- Digital signatures are used in various applications, including email authentication, document signing, and secure online transactions. They provide a high level of security and trust in digital communication.

Unit 3: Integrity, Authentication & Hash Functions

Authentication Protocols

Authentication protocols are sets of rules and procedures used to establish the identity of parties involved in a communication or transaction. These protocols ensure that the entities communicating with each other are who they claim to be.

- **Username and Password:** This is a basic authentication protocol where users provide a username and password to access a system. It's widely used but can be vulnerable to attacks like brute force and phishing.

Unit 3: Integrity, Authentication & Hash Functions

Authentication Protocols

- **OAuth:** OAuth (Open Authorization) is an authentication and authorization protocol used for secure access to resources on the internet. It's often used for single sign-on (SSO) and allows a user to grant third-party applications limited access to their resources.

- **OpenID Connect:** OpenID Connect is an identity layer built on top of OAuth 2.0. It provides authentication services and user information retrieval. It's commonly used for identity federation and SSO.

Unit 3: Integrity, Authentication & Hash Functions

Authentication Protocols

- **SAML (Security Assertion Markup Language):** SAML is an XML-based authentication and authorization protocol used for exchanging authentication and authorization data between parties. It's prevalent in enterprise environments for SSO (Single Sign On) and identity federation.
- **RADIUS (Remote Authentication Dial-In User Service):** RADIUS is a network protocol used for authenticating and authorizing remote users. It's commonly used in network access, such as VPNs and Wi-Fi authentication.

Unit 3: Integrity, Authentication & Hash Functions

Digital Signature Standard

The Digital Signature Standard (DSS) is a set of specifications and guidelines for digital signatures established by the National Institute of Standards and Technology (NIST) in the United States. DSS outlines the algorithms and cryptographic mechanisms that should be used to create and verify digital signatures.

Unit 3: Integrity, Authentication & Hash Functions

Digital Signature Standard

- **Algorithms:** DSS specifies the use of specific cryptographic algorithms for digital signatures. It mandates the use of the Digital Signature Algorithm (DSA) for creating digital signatures.
- **Key Lengths:** DSS provides guidelines for key lengths to ensure the security of digital signatures. These guidelines evolve over time as computing power and cryptographic knowledge advance.
- **Hash Functions:** DSS specifies the use of specific hash functions in combination with the DSA algorithm to create secure digital signatures.

Unit 3: Integrity, Authentication & Hash Functions

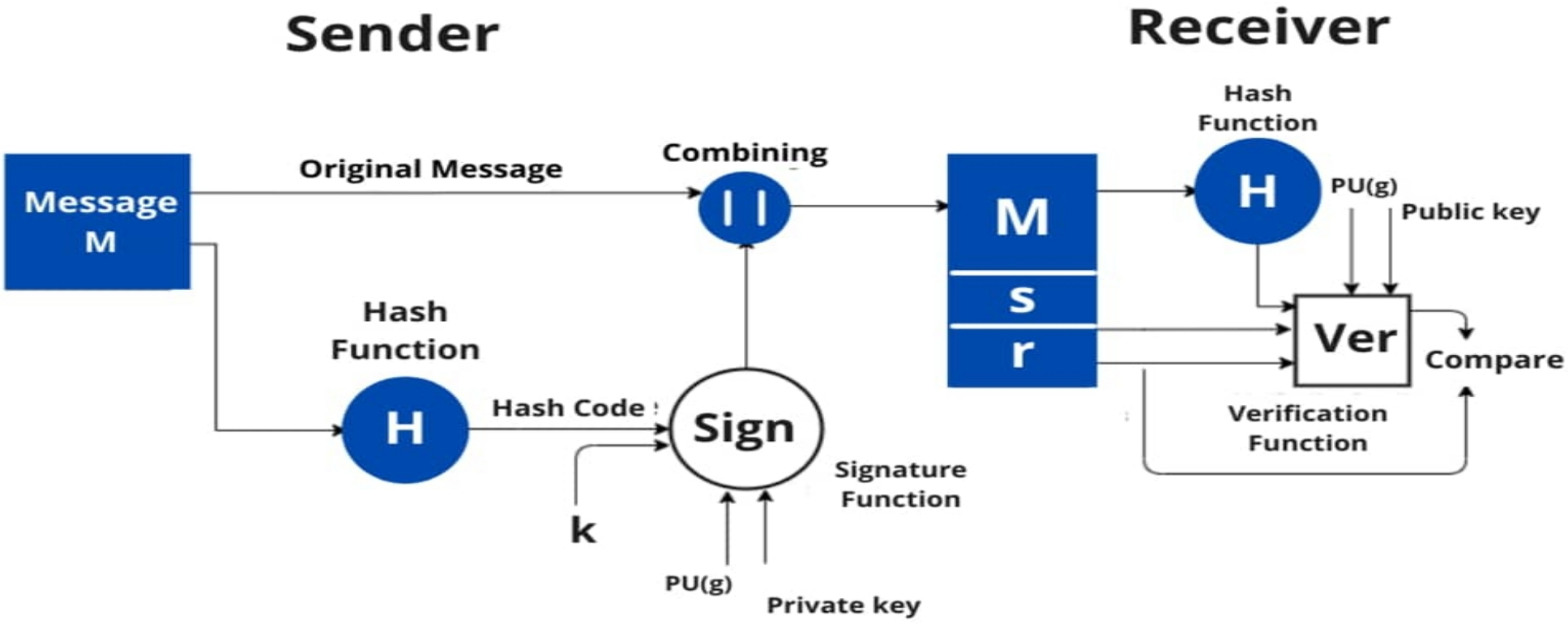
Digital Signature Standard

- **Compliance:** In many cases, government agencies and organizations handling sensitive data are required to comply with the Digital Signature Standard to ensure the security and legal validity of digital signatures (UIDAI, NIELIT, Driving License).

  DSS ensures that digital signatures created and verified using the standard are secure and reliable, making them suitable for various applications, including government, finance, and legal sectors.

Unit 3: Integrity, Authentication & Hash Functions

Digital Signature Standard



Unit 3: Integrity, Authentication & Hash Functions

Digital Signature Standard

- **Sender Side:** In DSS Approach, a hash code is generated out of the message and following inputs are given to the signature function –

1. The hash code.
2. The random number ‘k’ generated for that particular signature.
3. The private key of the sender i.e., PR(a).
4. A global public key(which is a set of parameters for the communicating principles) i.e., PU(g).

Digital Signature Standard

These input to the function will provide us with the output signature containing two components – ‘s’ and ‘r’. Therefore, the original message concatenated with the signature is sent to the receiver.

Unit 3: Integrity, Authentication & Hash Functions

Digital Signature Standard

- **Receiver Side:** At the receiver end, verification of the sender is done. The hash code of the sent message is generated. There is a verification function which takes the following inputs –

1. The hash code generated by the receiver.
2. Signature components ‘s’ and ‘r’.
3. Public key of the sender.
4. Global public key.

Digital Signature Standard

The output of the verification function is compared with the signature component ‘r’. Both the values will match if the sent signature is valid because only the sender with the help of it private key can generate a valid signature.

Unit 3: Integrity, Authentication & Hash Functions

How Public Key & Private key is used in Digital Signature?

Sender's Public Key	Used by receiver for sender's identity verification (Digital signature verification)
Sender's Private Key	Used by Sender for generating digital signature
Receiver's Private Key	
Receiver's Public Key	Used by sender for Data Encryption
Receiver's Private Key	Used by receiver for Data Decryption

Unit 3: Integrity, Authentication & Hash Functions

Digital Signature Standard

In summary, digital signatures and authentication protocols are fundamental components of information security. Digital signatures provide a means to verify the authenticity and integrity of digital messages, while authentication protocols establish the identity of parties involved in communication. The Digital Signature Standard provides guidelines for secure digital signature implementations. Together, these elements contribute to secure and trustworthy digital communication and transactions.

Unit 3: Integrity, Authentication & Hash Functions

Digital Signature Algorithm

- Here's a concise overview of commonly used digital signature algorithms:

- **RSA:** Based on prime numbers, widely supported.
- **DSA (Digital Signature Algorithm):** Used in government applications.
- **ECDSA (Elliptic Curve Digital Signature Algorithm):** Efficient and secure with shorter keys.

Unit 3: Integrity, Authentication & Hash Functions

Digital Signature Algorithm

- **EdDSA (Edwards-curve Digital Signature Algorithm):** Known for efficiency and security.
- **Schnorr Signatures:** Simple and secure.
- **GOST R 34.10:** Russian algorithm based on elliptic curves.
- **LMS (Leighton-Micali Signature):** Post-quantum secure.
- **XMSS (Extended Merkle Signature Scheme):** Post-quantum secure, hash-based.